

# Sistemas Distribuídos e Computação em Nuvem

## Aula 8 — Mensageria, eventos e API Gateway

C2 · Coordenação e consistência · 24/09/2026 · Lança o C2.A2

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

## Onde estamos — começa a C2

---

### C1 · Fundamentos e comunicação (1–7)

Sockets, concorrência, gRPC, REST, IA como serviço.

### C2 · Coordenação e consistência (8–12)

**Mensageria**, relógios lógicos, **CAP**, **Raft**, resiliência.

### C3 · Nuvem, implantação e segurança (13–18)

Containers, nuvem, serverless, observabilidade, segurança.

 **Aula 8** — a **fila** que o seu worker já usava, agora com a teoria.

## Retomada — fechamento da C1

 A C1 acabou: prova C1.A1 feita e trabalho C1.A2 entregue.

- No **C1.A2**, o seu serviço já enfileirava tarefas e um **worker** processava em segundo plano — mas você usou a fila **sem a teoria**.
- Hoje entendemos **por que** ela existe e **o que** ela resolve.

 Esta aula também **lança o C2.A2** — RAG Distribuído em Microserviços (kit novo). A mensageria de hoje é a **espinha** dele.

## Objetivos desta aula

---

Ao final, você será capaz de:

1. **Diferenciar** comunicação **síncrona** de **assíncrona** e saber quando usar cada uma.
2. **Processar** inferências de IA de forma **assíncrona** com **fila** e **worker**.
3. **Explicar** o papel de um **API Gateway** numa arquitetura de microsserviços.

## Conceito 1/4 — Síncrono × Assíncrono

### Síncrono

- O cliente chama e **fica esperando** a resposta.
- Simples, mas ele **trava** enquanto espera.
- Se muitos esperam ao mesmo tempo, o serviço **satura**.

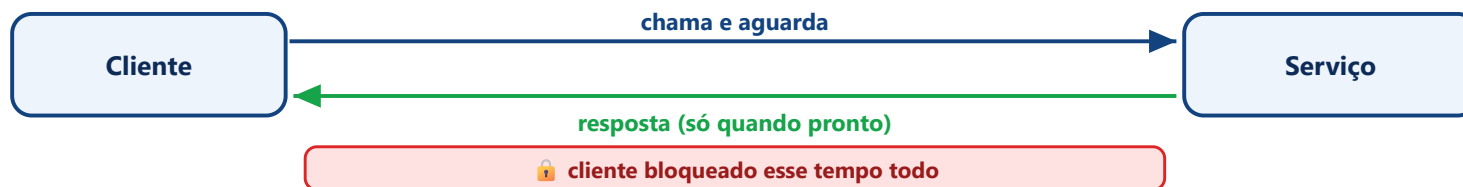
### Assíncrono

- O cliente **entrega a tarefa** e recebe **na hora** um **comprovante (id)**.
- Vai **buscar o resultado depois**.
- Quebra o **vínculo temporal** entre pedido e processamento.

💡 Para uma inferência de IA que leva **segundos** (Aula 6), segurar o cliente é desperdício. O assíncrono é a resposta ao **cold start / latência** que você mediu.

## Síncrono × Assíncrono, visualmente

**SÍNCRONO** — o cliente fica travado esperando



**ASSÍNCRONO** — entrega, recebe um id e segue livre



💡 **Em miúdos:** síncrono é a **fila do caixa** (você espera parado). Assíncrono é a **senha da lanchonete**: você pede, pega a senha (o id) e vai sentar; quando fica pronto, chamam o número.

## Conceito 2/4 — A fila como amortecedor

- O mecanismo por trás do assíncrono é a **fila de mensagens**: um **produtor** coloca a tarefa; um **worker** (consumidor) a retira e processa.
- Palavra-chave: **desacoplamento** — quem produz **não precisa saber** quem consome, nem quando, nem **quantos** consumidores existem.

### Absorve picos

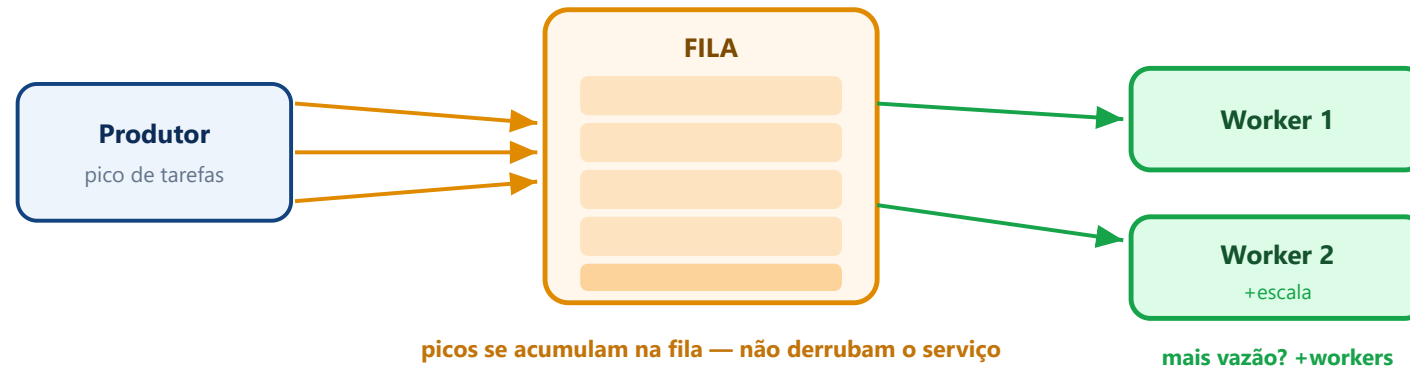
Chegou mais do que a capacidade? As tarefas **se acumulam na fila** em vez de **derrubar** o serviço.

### Escala simples

Precisa de mais vazão? **Suba mais workers** consumindo a mesma fila — **sem mudar** quem produz.

💡 É a **concorrência da Aula 3** num nível acima: em vez de threads no mesmo programa, **processos independentes** puxando da mesma fila.

## A fila como amortecedor, visualmente



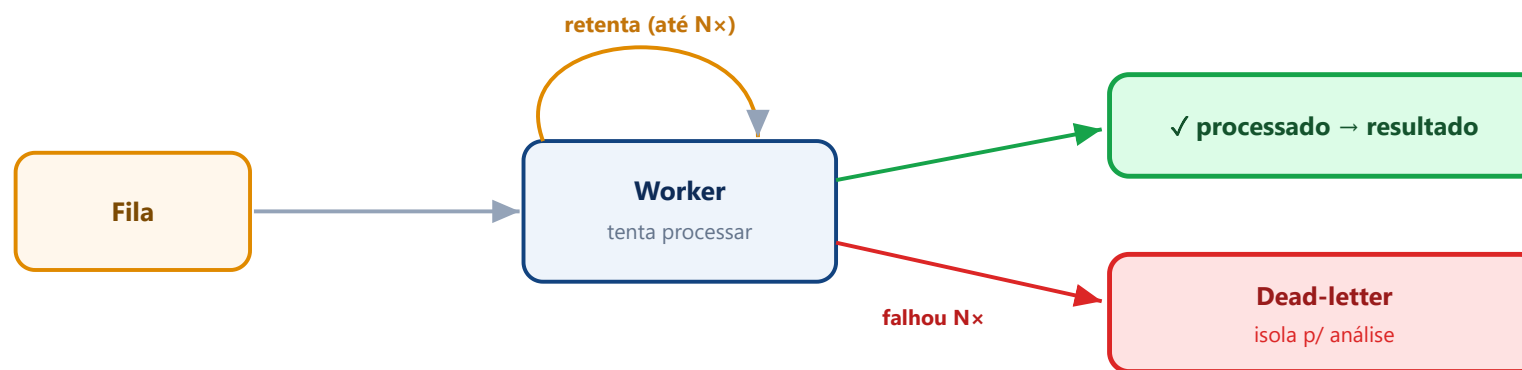
💡 **Em miúdos:** a fila é a **caixa de entrada** do restaurante. Os pedidos entram e ficam guardados; a cozinha (workers) puxa no ritmo dela. Movimento demais? **Coloque mais cozinheiros** na mesma caixa.

## Conceito 2/4 — Quando o processamento falha: dead-letter

- Uma tarefa pode dar **erro** (entrada inválida, dependência fora do ar). O que fazer?
- **Boa prática: reprocessar** algumas vezes; **persistindo** a falha, encaminhar a mensagem para uma **fila de descarte** — a **dead-letter**.

⚠ Os dois extremos são ruins: **tentar para sempre** trava o worker; **perder a mensagem** esconde o problema. A **dead-letter** isola a mensagem para análise e **mantém o fluxo principal saudável**.

## O que fazer quando falha — dead-letter



a mensagem problemática sai do caminho → o fluxo principal segue saudável

💡 **Em miúdos:** é a **bandeja de "pendências"** do balcão. Tentou atender, não deu depois de algumas vezes? Põe naquela bandeja para olhar com calma — **sem travar a fila** de quem vem atrás.

## Conceito 3/4 — Pub/sub e arquitetura de eventos

---

### Fila (work queue)

Cada mensagem é consumida por **um único** worker.  
Distribui **trabalho**.

### Pub/sub (publicar/assinar)

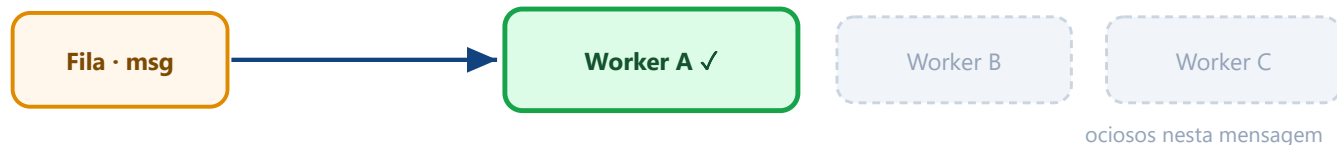
A mesma mensagem é entregue a **todos os interessados**. Distribui **informação**.

- O pub/sub é a base da **arquitetura orientada a eventos**, muito usada em nuvem.
- Ex.: um **pedido concluído** pode, ao mesmo tempo, **atualizar o estoque**, **notificar o cliente** e **alimentar um relatório**.

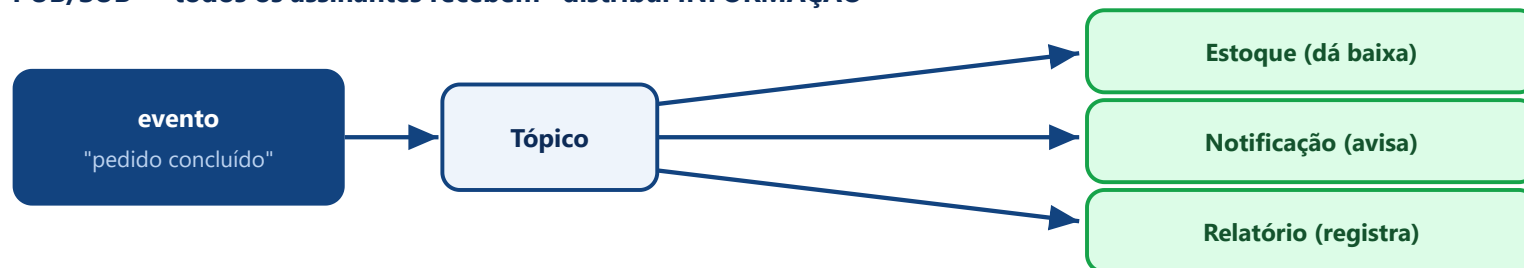
💡 Fila = **um** consumidor por mensagem. Pub/sub = **vários** assinantes na mesma mensagem.

## Fila × Pub/sub, visualmente

**FILA — 1 consumidor por mensagem · distribui TRABALHO**



**PUB/SUB — todos os assinantes recebem · distribui INFORMAÇÃO**



💡 **Em miúdos:** a fila é o **próximo, por favor!** — um atendente pega cada senha. O pub/sub é o **alto-falante do aeroporto:** um aviso, e **todo mundo interessado** ouve ao mesmo tempo.

## Conceito 4/4 — API Gateway: a porta única

- Com vários serviços, o cliente **não deve** conhecer o endereço de cada um.
- O **API Gateway** é a **porta de entrada única**: recebe todas as requisições e **encaminha ao serviço certo**.
- Centraliza o que é **comum a todos**: **autenticação**, **limite de requisições** (rate limit) e **log**. É nele que aplicaremos a **segurança** (C3).

⚠ **Versionamento**: **acrescentar** um campo à resposta é seguro; **remover/renomear** um campo **quebra** os clientes — exige publicar uma nova versão (ex.: `/v2`) e manter a antiga durante a transição.

# API Gateway — a porta única



o cliente conhece só uma porta — não o endereço de cada serviço

💡 **Em miúdos:** é a **recepção do prédio**. Ninguém entra direto na sala de cada setor: passa pela recepção, que **confere o crachá** (auth), controla o fluxo e **indica a sala certa**.

# Laboratório

O pipeline assíncrono — da fila em memória ao kit real

## Lab · Passo 1 — a fila como conceito ( `fila_demo.py` )

Sem Docker ainda: uma fila **em memória** com **3 workers** (threads da Aula 3):

```
import queue, threading
fila = queue.Queue(); resultados = {}; dead_letter = []

def worker(nome):
    while True:
        try: tarefa = fila.get(timeout=2)           # pega da fila (bloqueia)
        except queue.Empty: return
        try:
            resultados[tarefa["id"]] = processa(tarefa)    # a "inferência"
        except Exception:
            tarefa["tentativas"] += 1
            if tarefa["tentativas"] < 3: fila.put(tarefa)  # RETENTATIVA
            else: dead_letter.append(tarefa)              # DEAD-LETTER

for i in range(9): fila.put({"id": i, "tentativas": 0})  # o PRODUTOR
# 3 WORKERS dividem a carga (escalar = + workers na MESMA fila):
[threading.Thread(target=worker, args=(f"w{i}",), daemon=True).start() for i in range(3)]
```

## Lab · Passo 2 — rodar e observar

```
python fila_demo.py
# [produtor] enfileirou 9 tarefas
# [w2] processou tarefa 2      <- os 3 workers dividem a carga
# [w0] processou tarefa 0
# ...
# [w1] 99 -> DEAD-LETTER      <- a tarefa "envenenada" falhou 3x
# processadas: 8 | na dead-letter: 1
```

💡 Experimente: mude `range(3)` para **1 worker** e veja demorar mais; para **6 workers** e veja acelerar. É a **escalabilidade** da fila na prática — sem tocar no produtor.

## Lab · Passo 3 — o pipeline real, no kit da C1 (Redis)

O kit já traz a fila de verdade (**Redis**) em `app/fila.py`:

```
POST /predict → fila.enfileirar(texto) → devolve {"id": ...} (não espera!)
                | (Redis)
worker → fila.proxima_tarefa() → modelo.prever() → fila.guardar_resultado(id, ...)
GET /resultado/{id} → fila.buscar_resultado(id) → {"status": "pronto", ...}
```

```
cd sd-2026-2-kit-c1a2      # a pasta do kit — é AQUI que está o docker-compose.yml
docker compose up -d       # sobe o Redis (lê o docker-compose.yml deste diretório)
uvicorn app.api_rest:app   # a API (POST /predict, GET /resultado)
python -m app.worker       # o worker — suba VÁRIOS e veja dividir
```

 O `docker-compose.yml` está na **raiz do kit** (`sd-2026-2-kit-c1a2/`, ao lado de `app/`) — rode o `docker compose` **de dentro dessa pasta**. Erro *"no configuration file provided"* = diretório errado.

 São as **TAREFAS 1, 2 e 3** do C1.A2: `POST /predict` (enfileira), `GET /resultado/{id}` (consulta) e o worker **gravar o resultado**.

## Lançamento do C2.A2 — RAG Distribuído em Microsserviços

Começa o **C2.A2**: um **RAG** (busca + geração) dividido em **microsserviços** ( `ingestao` , `recuperacao` , `geracao` ), acessados por um **gateway**.

- **Clonar** o kit novo: `sd-2026-2-kit-c2a2`.
- **TAREFA 4 — mensageria**: um **novo worker** + `docker-compose.yml` (ao menos **uma etapa** do fluxo passa por **fila**).
- **Iniciar** o serviço de **ingestão/embeddings**.

 A mensageria de hoje é a **espinha** do C2.A2. Você já sabe o padrão: produtor → fila → worker, com **dead-letter** para falhas.

## Atividade para casa

1. **Complete o worker** do C1 (kit): **TAREFA 3** (gravar o resultado) e **TAREFA 5** (retentativa + dead-letter em vez de só registrar o erro).
2. **Teste sob carga:** dispare muitos `POST /predict` e confira que a fila **absorve** e os resultados saem pelo `GET /resultado/{id}`.
3. **Inicie o C2.A2:** clone `sd-2026-2-kit-c2a2` e ponha o **serviço de ingestão** no ar.

📌 **Entregar até a próxima aula:** worker com **retentativa + dead-letter** + repositório do **C2.A2** com a ingestão iniciada.

## ◆ Foco ENADE

---

O que costuma cair:

- Comunicação **síncrona** × **assíncrona** entre processos.
- **Middleware orientado a mensagens (MOM)**, **filas** e **pub/sub**.
- **Arquitetura orientada a eventos** e **desacoplamento**.
- **Microserviços**, **API Gateway** e **versionamento** de contratos.

**Termos-chave:** Fila · Produtor · Worker · Pub/sub · Dead-letter · API Gateway · Versionamento

💡 Papéis: **produtor** publica, **worker** consome. Fila distribui **trabalho**; pub/sub distribui **informação**.

## Questão de autoavaliação (estilo ENADE)

---

Sobre a diferença entre uma **fila de trabalho** e o modelo **publicar/assinar (pub/sub)**, assinale a correta.

- A) Na fila as mensagens são descartadas; em pub/sub, persistidas.
- B) Na fila, cada mensagem é consumida por **um único** consumidor; em pub/sub, a mesma mensagem é entregue a **todos os assinantes** interessados.
- C) A fila exige consistência forte e o pub/sub, consistência eventual.
- D) A fila só funciona com UDP e o pub/sub, apenas com TCP.
- E) Não há diferença prática entre os dois modelos.

## Resolução — alternativa B

---

- A **fila distribui trabalho**: um consumidor por mensagem.
- O **pub/sub distribui informação**: vários assinantes recebem a **mesma** mensagem.
- As demais inventam distinções que **não existem** (descarte, consistência, protocolo de transporte).

💡 É o seu `fila_demo.py` (um worker pega cada tarefa) contra o padrão de eventos (todos recebem).

## Fora da sala · Glossário

---

### Para estudar

- **Coulouris**, cap. 6 — Comunicação indireta (mensageria, pub/sub).
- Documentação do **Redis** (listas como fila) — a que o kit usa.
- Releia `app/fila.py` e `app/worker.py` do **kit da C1**.

### Glossário

- **Fila**: guarda tarefas até um consumidor processar.
- **Worker**: processo que retira e executa mensagens.
- **Pub/sub**: uma mensagem publicada chega a vários assinantes.
- **Dead-letter**: fila de descarte para o que falhou demais.
- **API Gateway**: porta única que encaminha aos serviços internos.

# Até a próxima aula

Complete o worker (retentativa + dead-letter) e inicie o C2.A2.

← Anterior

Aula 7 · Revisão + prova C1

≡ Índice

todas as aulas

Próxima aula →

Aula 9 · Relógios lógicos